

Apuntes Semana 11 Clase II

Hans Fernández Murillo

2016193340

Tecnológico de Costa Rica

Curso: IC6200 Inteligencia Artificial

Fecha: 07 de Mayo de 2026

Abstract—En esta clase se vieron los autoencoders, arquitectura muy utilizada en procesamiento de imágenes. Entre los contenidos del apunte están: usos, partes del autoencoder, hiperparámetros y tipos de autoencoder.

Index Terms—autoencoder, decoder, encoder, latent space, convolutional networks

I. INTRODUCCIÓN

Al inicio de la clase los grupos de proyecto presentaron avances del Proyecto I. Se acordó mover la fecha de entrega al Domingo 10 de Mayo a las 7:00pm con las siguientes condiciones:

- 1) Informe tiene que presentar sección de State of the Art.
- 2) Mostrar gráficas comparativas de Training y Validation de los modelos (overfitting) WandB.
- 3) Análisis de matriz de confusión vs los resultados obtenidos con el mobile.

Se mencionó además que la semana 12 sería de revisiones y no hubo clases. Deben estar todos los integrantes del grupo en la defensa. La defensa será solo en el celular, pero también se debe llevar el código del APK y notebook.

II. AUTOENCODER

El Autoencoder es una arquitectura relacionada a procesamiento de imágenes. Hasta ahorita hemos visto que tenemos una red capaz de procesar una imagen a través de una CNN. Entre algunos ejemplos de arquitecturas CNN están LeNet, Alexnet, Resnet, Densenet, Mobilenet, VGG. Estos modelos consisten en la extracción de características de una imagen, dando como resultado un vector que puede ser utilizado por un clasificador.

El Autoencoder por su parte no clasifica, sino que reconstruye imágenes.

A. Unsupervised Learning

Muchas de las lecturas mencionan que los autoencoders utilizan aprendizaje no supervisado. Esto quiere decir que no utiliza etiquetas y el resultado se compara contra la misma entrada X . Otra forma de verlo es que la entrada es la misma etiqueta. La arquitectura se puede resumir de la siguiente forma: Al pasar una imagen de entrada por una CNN que la reduzca a un vector, reconstruir esa imagen utilizando ese vector de características. Para efectos de la clase se utiliza el ejemplo de imágenes, sin embargo esto se puede aplicar a otros datos como texto. A pesar de que técnicamente es un

modelo de aprendizaje no supervisado, sí se utiliza la entrada como etiqueta para supervisar el aprendizaje.

B. Cómo se ve un autoencoder

Se tiene una entrada que se pasa por un encoder y se transforma a un vector reducido con la información semántica de la entrada. Este se llama vector latente o espacio latente. Después, el vector latente pasa por un proceso de Decoder que es lo que reconstruye los datos. Otro ejemplo de autoencoder, es el que genera un μ y un σ con los que se puede generar un espacio latente continuo para la generación de imágenes. Esto puede generar imágenes nunca antes vistas por el modelo.

Las partes de este modelo en orden de ejecución son:

- 1) Encoder
- 2) Cuello de botella, vector latente, espacio latente
- 3) Decoder

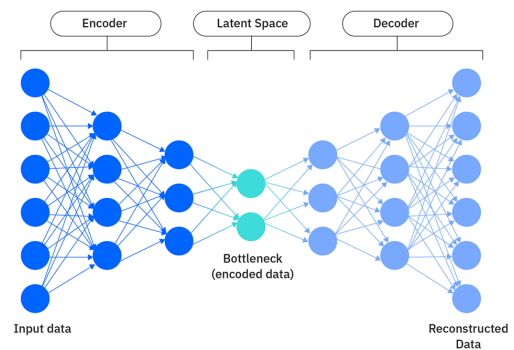


Fig. 1. Arquitectura Autoencoder

III. TAREAS QUE RESUELVE

A. Reducción de dimensionalidad

Para este propósito, removemos el Decoder. El punto es tener un Encoder el cuál reduce los datos a un vector latente. Algunos ejemplos prácticos son: reducción de espacio de los datos, comparación de imágenes. La calidad del vector producido por estos modelos es mejor que un algoritmo como PCA.

B. Detección de anomalías

Si se entrena el modelo teniendo en cuenta que los datos del entrenamiento poseen la cualidad de correctitud y luego se le

pasa como entrada al modelo datos que presentan anomalías comparado a dicha correctitud, el vector latente producido debería de ser de baja calidad y por lo tanto la salida del Decoder será de baja calidad. Esto implica que al comparar el resultado $\hat{y}$ con la entrada $(x - \hat{y})^2$ se obtiene un resultado anómalo. Es necesario definir el umbral en el cuál un resultado de la función de loss se considera anómalo. Puede ser usado para detectar transacciones fraudulentas.

C. Procesamiento de imágenes

Para compresión de imágenes, reducción de ruido, super resolution. Este caso tiene una consideración interesante: de entrada se requiere una imagen de buena calidad y su equivalente en baja calidad (por ejemplo: ruido agregado o menor resolución). De entrada al encoder, se le pasa la imagen de baja calidad, pero al dar la salida el decoder, esta se compara con la imagen de entrada de buena calidad.

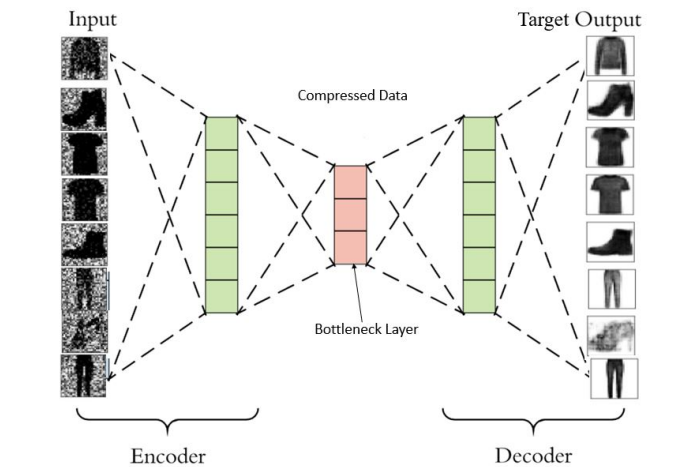


Fig. 2. Procesamiento de imágenes

IV. ENCODER

Condensa la imagen a un vector y aprende esa distribución del vector. Este proceso se le conoce como downsampling. Conjunto de bloques convolucionales seguidos de módulos de pooling. Encargada de extraer las características principales de la imagen de entrada y comprimir la información.

Expectativa: aprender información importante de la entrada. Produce una representación latente.

V. CUELLO DE BOTELLA

Resultado del proceso de encoding o proceso de convolución. Es el espacio latente, la representación latente. De la calidad de este espacio viene la calidad del resultado del proceso de reconstrucción de los datos. Llama la atención ya que debe ser la parte más "pequeña" del modelo. Reduce o restringe el flujo de información proveniente del encoder hacia el decoder. Lo que se espera visualmente de este proceso es que los datos se distribuyan de forma ordenada según la entrada y el avance del entrenamiento.

VI. DECODER

Conjunto de convoluciones que hacen upsampling. Reconstruye la imagen basada en el vector latente. Entre más grande sea el vector latente más fácil es para el decoder producir un resultado de calidad, por lo que se debe mantener un balance para que no sea muy extenso ni muy reducido. El método en python que hace proceso inverso de convolución se llama ConvTranspose2d, de la librería PyTorch.

VII. HIPERPARÁMETROS A CONSIDERAR

- **Tamaño de la codificación (vector latente):** decide cuánto se comprimen los datos. Entre más pequeño sea el vector, mejor comprensión según la calidad. Sin embargo entre más pequeño sea el vector se tienen menos datos para el proceso de reconstrucción.
- **Número de capas:** se decide por el diseñador del modelo. Definen la profundidad del encoder y el decoder. Siguen la misma regla: mayor número de capas equivale a un modelo más complejo pero mayor tiempo para ejecutar. Menor número de capas equivale a un modelo más rápido pero a la vez sencillo.
- **Función de reconstrucción:** depende de la entrada y salida esperada. Las básicas que se mencionan son MSE (Mean square error) donde se compara pixel por pixel las imágenes. Binary Cross Entropy, al procesar imágenes en blanco y negro se puede reducir a un problema de probabilidades donde los valores están en el rango de $[0, 1]$.

VIII. TIPOS DE AUTOENCODER

A. Undercomplete Autoencoder

Autoencoder vanilla. Toma una imagen de entrada e intenta predecir la misma imagen de salida. La dimensión del espacio latente es menor que el espacio de entrada. Disminuye la entrada hasta producir el cuello de botella, de ahí se pasa al decoder para reconstruir la imagen.

B. Sparse Autoencoder

Evita la reducción de dimensionalidad en las capas ocultas. Intenta que la arquitectura siempre sea la misma y no reducir las dimensiones. Selectivamente activa neuronas o regiones de la red dependiendo de la entrada.

C. Denoising Autoencoder

No es una arquitectura pero se considera una categoría de autoencoder debido a la cantidad de papers relacionados al tema. Se encarga de remover el ruido de una imagen. No tiene la imagen de entrada como salida realmente, ya que la imagen de entrada es la imagen sin ruido.

D. Variational Autoencoder

Autoencoders cuyo propósito es representar un espacio latente. Útil para cuando se requiere representar un espacio que no es continuo o que es difícil de interpolar. Interpolar se le conoce al proceso de estimar valores desconocidos dentro

de un conjunto de datos conocidos. Expresa los valores del vector latente como una distribución de probabilidad.

A partir de los Variational Autoencoders empieza la IA Generativa.

Para este tipo de autoencoders:

- Codificador produce dos vectores para hacer la distribución: una media μ y una desviación estándar σ . Se espera que imágenes que pertenecen a una misma clase produzcan la misma μ y σ .
- Reparametrización: se toman muestras de μ y σ para producir el vector latente. A partir de μ y σ se aproxima que sigan una distribución normal $N(0, 1)$.
- Se toma el vector latente y se alimenta al decoder.
- Se tiene una función de pérdida compuesta: Disminuir la suma de Binary Cross Entropy (BCE) y Kullback-Leibler Divergence (KLD). KLD compara dos distribuciones y nos dice qué tanto se parecen.

En la figura 3 se puede observar un ejemplo de cómo se pueden representar el resultado de la reconstrucción en un espacio continuo.

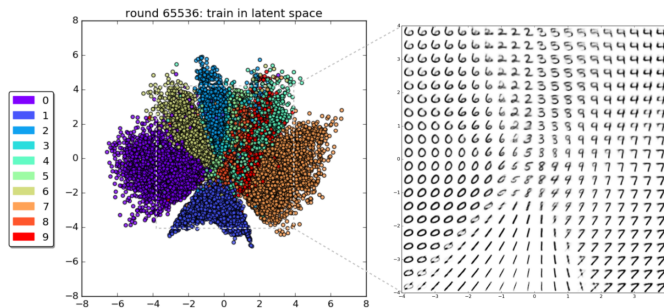


Fig. 3. Procesamiento de imágenes